

D MART E-COMMERCE MANAGEMENT SYSTEM

1. Creation of ORDERS Table

The ORDERS table is created to store the details of customer orders. It contains information such as Order ID, Customer ID, Order Date, Total Amount, and Order Status. This table helps to maintain and manage order-related information in the e-commerce system.

```
Connected.  
SQL> CREATE TABLE ORDERS (Order_ID VARCHAR2(10), Customer_ID VARCHAR2(10), Order_Date DATE,  
2 Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));  
Table created.
```

2. Creation of ORDER_DETAILS Table

The ORDER_DETAILS table is created to store detailed information about each order. It contains Order Detail ID, Order ID, Product ID, Quantity, and Price. This table helps to maintain the products included in each customer order.

```
SQL> CREATE TABLE ORDER_DETAILS (Order_Detail_ID VARCHAR2(10), Order_ID VARCHAR2(10),  
2 Product_ID VARCHAR2(10), Quantity NUMBER, Price NUMBER(10,2));  
Table created.
```

3. Primary Key for ORDERS

A primary key is added to the ORDERS table using Order ID. It uniquely identifies each order and prevents duplicate order records.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);  
Table altered.
```

4. Primary Key for ORDER_DETAILS

A primary key is added to the ORDER_DETAILS table using Order Detail ID. It uniquely identifies each order detail record.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_OrderDetails PRIMARY KEY (Order_Detail_ID);  
Table altered.
```

5. Foreign Key for Customer

A foreign key relationship is created between the ORDERS and CUSTOMER tables using Customer ID. This connects each order with the customer who placed it.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_Orders_Customer FOREIGN KEY (Customer_ID)
  2 REFERENCES CUSTOMER(Customer_ID);
Table altered.
```

6. Foreign Key for Product

A foreign key relationship is created between the ORDER_DETAILS and PRODUCT tables using Product ID. This connects each order detail with the corresponding product.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Product FOREIGN KEY (Product_ID)
  2 REFERENCES PRODUCT(Product_ID);
Table altered.
```

7. NOT NULL Constraint for Order Date

The Order Date field is made mandatory so that every order must have a valid order date. This helps to maintain complete order information.

```
SQL> ALTER TABLE ORDERS MODIFY Order_Date NOT NULL;
Table altered.
```

8. Check Constraint for Total Amount

A check constraint is applied to Total Amount to ensure that the value is not negative. This maintains valid order amount information.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);
Table altered.
```

9. Check Constraint for Order Status

A check constraint is used to restrict the order status to valid values such as Pending, Shipped, and Delivered. This helps to maintain consistent order status information.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Status CHECK (Order_Status IN ('Pending', 'Shipped',  
2 'Delivered'));  
Table altered.
```

10. Check Constraint for Quantity

A check constraint is applied to Quantity to ensure that the quantity is greater than zero. This prevents invalid product quantities from being entered.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_Qty CHECK (Quantity > 0);  
Table altered.
```

11. Check Constraint for Price

A check constraint is applied to Price to ensure that the product price is not negative. This maintains valid pricing information.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_OD_Price CHECK (Price >= 0);  
Table altered.
```

12. Displaying Order Details

The order details are retrieved from the ORDER_DETAILS table. This helps to view the products and other information related to customer orders.

```
SQL> SELECT * FROM ORDER_DETAILS ;
```

ORDER_DETA	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
-----	-----	-----	-----	-----
OD101	0105	101	32	2500
OD102	0102	102	32	3500
OD103	0103	103	55	67000
OD104	0104	104	77	77000
OD105	0105	104	55	55500

13. Displaying Customer and Order Details

Customer, order, and product information are combined to display complete order details. The result includes customer information, order date, product ID, quantity, price, and subtotal.

```
SQL> INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE','YYYY-MM-DD'),&TOTAL_AMOUNT,'&ORDER_STATUS');
Enter value for order_id: 0101
Enter value for customer_id: 101
Enter value for order_date: 2007-05-30
Enter value for total_amount: 20000
Enter value for order_status: Shipped
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE','YYYY-MM-DD'),&TOTAL_AMOUNT,'&ORDER_STATUS')
new 1: INSERT INTO ORDERS VALUES ('0101', 101, TO_DATE('2007-05-30','YYYY-MM-DD'),20000,'Shipped')

1 row created.

SQL> /
Enter value for order_id: 0102
Enter value for customer_id: 102
Enter value for order_date: 2003-05-21
Enter value for total_amount: 50000
Enter value for order_status: Pending
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE','YYYY-MM-DD'),&TOTAL_AMOUNT,'&ORDER_STATUS')
new 1: INSERT INTO ORDERS VALUES ('0102', 102, TO_DATE('2003-05-21','YYYY-MM-DD'),50000,'Pending')

1 row created.

SQL> /
Enter value for order_id: 0103
Enter value for customer_id: 103
Enter value for order_date: 2002-09-13
Enter value for total_amount: 8700
Enter value for order_status: Shipped
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE','YYYY-MM-DD'),&TOTAL_AMOUNT,'&ORDER_STATUS')
new 1: INSERT INTO ORDERS VALUES ('0103', 103, TO_DATE('2002-09-13','YYYY-MM-DD'),8700,'Shipped')

1 row created.

SQL> /
Enter value for order_id: 0104
Enter value for customer_id: 104
Enter value for order_date: 2004-07-28
Enter value for total_amount: 5200
Enter value for order_status: Shipped
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE','YYYY-MM-DD'),&TOTAL_AMOUNT,'&ORDER_STATUS')
new 1: INSERT INTO ORDERS VALUES ('0104', 104, TO_DATE('2004-07-28','YYYY-MM-DD'),5200,'Shipped')

1 row created.

SQL> /
Enter value for order_id: 0105
Enter value for customer_id: 105
Enter value for order_date: 2008-12-13
Enter value for total_amount: 9000
Enter value for order_status: Pending
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE','YYYY-MM-DD'),&TOTAL_AMOUNT,'&ORDER_STATUS')
new 1: INSERT INTO ORDERS VALUES ('0105', 105, TO_DATE('2008-12-13','YYYY-MM-DD'),9000,'Pending')

1 row created.
```

14. Displaying Customer Order Summary

The system displays the customer name, order ID, order date, and total amount. This provides a simple summary of the orders placed by each customer.

```
SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.CUSTOMER_NAME,
  4     O.ORDER_ID,
  5     O.ORDER_DATE,
  6     OD.PRODUCT_ID,
  7     OD.QUANTITY,
  8     OD.PRICE,
  9     OD.QUANTITY * OD.PRICE AS SUBTOTAL
10 FROM CUSTOMER C
11 JOIN ORDERS O
12     ON C.CUSTOMER_ID = O.CUSTOMER_ID
13 JOIN ORDER_DETAILS OD
14     ON O.ORDER_ID = OD.ORDER_ID
15 ORDER BY C.CUSTOMER_ID, O.ORDER_DATE;
```

CUSTOMER_ID

CUSTOMER_NAME

ORDER_ID	ORDER_DAT	PRODUCT_ID	QUANTITY	PRICE	SUBTOTAL
102					
BEC					
0102	21-MAY-03	102	32	3500	112000

103					
BECBEC					
0103	13-SEP-02	103	55	67000	3685000

CUSTOMER_ID

CUSTOMER_NAME

ORDER_ID	ORDER_DAT	PRODUCT_ID	QUANTITY	PRICE	SUBTOTAL
104					
REBECCA PATRICIA ARMSTRONG					
0104	28-JUL-04	104	77	77000	5929000

15. Calculating Total Quantity of Products

The system calculates the total quantity ordered for each product. This helps to understand how many units of each product have been ordered.

```
SQL> SELECT P.PRODUCT_NAME,SUM(O.QUANTITY) AS TOTAL_QUANTITY FROM PRODUCT P JOIN  
2 ORDER_DETAILS O ON P.PRODUCT_ID=O.PRODUCT_ID GROUP BY P.PRODUCT_NAME;
```

PRODUCT_NAME	TOTAL_QUANTITY
Red Grapes	132
Broccoli	55
Kiwi Fruit	32
Quinoa	32

Conclusion

The D Mart E-Commerce Management System uses database tables, keys, relationships, and constraints to manage customer orders and product details effectively. These operations help maintain accurate, consistent, and organized e-commerce data.